

SOFTWARE REQUIREMENTS SPECIFICATION

BookEase

Healthcare Booking Platform

July 12, 2026

Author	Parana Manage Udesha Rashmi
Project	BookEase Healthcare Booking Platform
Document Type	Software Requirements Specification (SRS)

Table of Contents

Table of Contents	2
1. Introduction.....	3
1.1 Purpose	3
1.2 Scope.....	3
1.3 Definitions	3
2. System Overview.....	3
3. System Actors	3
4. Functional Requirements	4
4.1 Guest User	4
4.2 Registered User	4
4.3 Admin.....	4
5. Use Case Example.....	4
Use Case: Create Booking	4
6. Non-Functional Requirements.....	5
7. User Interface Requirements.....	5
8. System Architecture	5
9. Business Rules	5
10. Database Design.....	5
User	5
Service.....	6
Booking	6
11. Booking Statuses	6
12. API Behavior	6
13. Constraints.....	7
14. Assumptions	7
15. Future Enhancements	7
16. Further Reading.....	7

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the BookEase Healthcare Booking Platform. The system enables patients to browse healthcare services and submit appointment requests, while providing administrative tools for managing services and bookings.

1.2 Scope

BookEase is a full-stack web application that allows:

- Patients to explore healthcare services
- Users to book appointments online
- Admins to manage services and bookings
- Guests to submit booking requests without authentication

The system is designed for simplicity, usability, and scalability.

1.3 Definitions

- Guest User: A user who can create bookings without logging in
- Registered User: A user with an account who can manage bookings
- Admin: A system administrator with full access to manage services and bookings

2. System Overview

BookEase follows a client-server architecture:

- Frontend: Next.js (React-based UI)
- Backend: NestJS, REST API
- Database: PostgreSQL with Prisma ORM

3. System Actors

- Guest User
- Registered User
- Admin

4. Functional Requirements

4.1 Guest User

- Browse healthcare services
- Submit appointment requests

4.2 Registered User

- Register and log in
- View available services
- Create bookings
- View personal booking history
- Cancel bookings

4.3 Admin

- Create, update, and delete services
- View all bookings
- Update booking statuses
- Manage system data

5. Use Case Example

Use Case: Create Booking

Actor: Guest User / Registered User

Precondition: Service must be active

Flow:

1. User selects a healthcare service
2. User chooses a date and time
3. User enters personal details
4. System validates input
5. Booking is created with PENDING status

6. Non-Functional Requirements

- The system should handle multiple concurrent users efficiently
- API response time should be under 2 seconds
- The system must ensure secure authentication using JWT
- The application should be responsive across devices (mobile & desktop)
- The system should maintain data integrity and validation

7. User Interface Requirements

- Clean and intuitive user interface
- Responsive design for mobile and desktop
- Simple and fast booking workflow
- Admin dashboard for managing services and bookings

8. System Architecture

- Client Layer: Next.js frontend
- Application Layer: NestJS backend API
- Data Layer: PostgreSQL database with Prisma ORM

9. Business Rules

- A booking must belong to an existing service.
- Booking dates cannot be in the past.
- Canceled bookings cannot be marked as completed.
- Only authenticated users can manage services.
- Customers can create bookings without authentication.

10. Database Design

User

Field	Description
id	Unique identifier for the user
name	Full name of the user
email	User's email address (used for login)
password	Hashed password for authentication
role	Role of the user (e.g., USER, ADMIN)

Service

Field	Description
id	Unique identifier for the service
title	Name of the healthcare service
description	Detailed description of the service
doctorName	Name of the doctor providing the service
price	Cost of the service
isActive	Indicates whether the service is currently bookable

Booking

Field	Description
id	Unique identifier for the booking
customerName	Name of the customer making the booking
customerEmail	Email address of the customer
customerPhone	Phone number of the customer
serviceId	Reference to the booked service
bookingDate	Date of the appointment
bookingTime	Time of the appointment
status	Current status of the booking

11. Booking Statuses

- PENDING
- CONFIRMED
- CANCELLED
- COMPLETED

12. API Behavior

- All responses follow a standard JSON format
- Errors include message and HTTP status codes
- Authentication is handled using JWT tokens

13. Constraints

- Requires Node.js runtime environment
- PostgreSQL database must be available
- Internet connection is required
- Environment variables must be configured

14. Assumptions

- The system does not include payment processing
- Real-time doctor availability is not implemented
- Email/SMS notifications are not included in the current version

15. Future Enhancements

- Email and SMS notifications
- Payment integration
- Doctor availability scheduling
- Admin analytics dashboard
- Booking rescheduling feature

16. Further Reading

For further implementation details, setup instructions, and technical documentation, refer to the project's GitHub repository README:

<https://github.com/UdeshaRashmi/bookease-api/blob/main/README.md>